Researchly Trend Tracker

Project Documentation & Analytical Report
Prepared by: Lehlohonolo Mofokeng Module: ITNPA4-34

Link to GitHub repo : https://github.com/Hlonii-LM/ITNPA4-34-Assessments-2026-

1. Introduction & Overview

The aim of this project was to build a prototype tool that helps researchers quickly scan and analyse papers. I split the project into two parts. The first part, Questions 1 and 2 uses machine learning to classify abstracts into AI or Classical economics. The second part, Questions 3 and 4 focuses on text-to-text summarising, topic extraction and topic visualisation. Finally I created a web application for the classification model so that people can actually use it.

---

2. Question 1 & 2: Abstract Classification

2.1 Data Cleaning

The data set had 1786 rows but many rows were removed and there were null values. The biggest challenge was the writing style. Abstracts in journals are full of noise, such as DOIs copyright notices, mathematical equations and citation brackets.

I built a text denoising pipeline that uses regex to remove this noise. I removed URLs, emails, boilerplate phrases like "JEL classification" and NBER blurbs. I also removed English stopwords but kept negations such, as "not" and "never" because they affect meaning. I applied lemmatisation, which gave me an usable vocabulary.

2.2 Feature Engineering

TF-IDF (Term Frequency-Inverse Document Frequency) is used for the features. That is why I looked at unigrams and bigrams words and two-word phrasesso the model could pick up specific terms or expressions like the "phillips curve". I also used sublinear_tf to make sure that words appearing a million times don't overpower the signal in the data.

2.3 Baseline Model

I already have a Logistic Regression model. The dataset is heavily skewed 81% Classical and 19% AI so I made sure to set class_weight='balanced'. This helps stop the model from predicting "Classical" all the time, which would be a kind of cheating.

How did it do?

• Training Accuracy: 98.57%

• Testing Accuracy: 91.67% ROC-AUC: 0.956

The model did very well on the training data.. The drop to 91.67%, on the test set is actually a good sign. It means the model isn't overfitting much. Looking at the confusion matrix it

correctly identified 40 out of 48 AI papers, which's a solid result. This gives a starting point for the next phase.

2.4 Model Enhancements (Question 2)

Four other models. Naïve Bayes, Linear SVC, Random Forest and HistGradientBoosting. Were trained to outperform the baseline. Among them Linear SVC performed the best achieving an accuracy of 96.4%. This model works with sparse text data, which made it a strong choice for this task.

To improve the results further I did two things. First I applied Chi-Square Feature Selection to reduce the number of features. The original dataset had 8,000 features. I kept only the top 2,000 that were most relevant. This helped focus on the informative parts of the text.

Second I used a Soft-Voting Ensemble. I combined the predictions from Linear SVC, Logistic Regression and HistGradientBoosting. In voting instead of just taking the majority vote the models combine their predicted probabilities. This approach gave a test accuracy of 96.9% and an ROC-AUC score of 0.981. The ensemble works well because each model tends to make mistakes. By combining them the errors cancel out. The overall performance improves.

---

3. Question 3: Corpus Summarization

3.1 The Data

I also unzipped the file. I loaded the five articles into a DataFrame. Applied the same denoising process used earlier. On average about 43% of the original text remained after removing headers, references and other noise. That means more than half of the text was irrelevant content..

3.2 Summarizing the First Article

Briefly the first article used Sentence-Transformers and a Semantic Extractive method. I took each sentence calculated a centrality score by measuring how similar each sentence was to the rest of the document and then selected the six sentences. I made sure no sentence was repeated so that the summary would not say the thing twice. I also created a similarity heat map and a bar chart. These visualizations were very helpful. They showed clearly why those six sentences were the central, to the article’s argument. The graphs made it easier to understand how the sentences connected and how they built the core message.

3.3 Aggregate Summarization

In the 4 articles I used LDA (Latent Dirichlet Allocation) to find the main topics. I tried numbers of topics and looked at a coherence score and k=4 was the best. The themes were the same as the 4 papers:

1. Reflexivity (insider/outsider positioning)

2. This includes reliability (Cronbachs alpha and psychometrics)

3. Ethical implications of technology and the internet (Denzins framework and validity)

4. Developmental Research (designs.

I did this by making word clouds and a document-topic mixture heatmap. The heat map looked almost diagonal which meant that all the papers were focused on one theme.

---

4. Question 4: Topic Modeling & Deployment

4.1 LDA Visualization

I re-ran the LDA on the collection and made horizontal bar graphs of the 12 most common words for each topic. The distribution was clear and not messy. The total sense is the topic-to-doc heatmap which showed that most papers focused on one topic. For the Mixed Methods paper it split the weight between two topics: Reflexivity and Triangulation.

4.2 Interactive pyLDAvis Map

I made an interactive Intertopic Distance Map using pyLDAvis. This was definitely the part of the project. It shows the topics on a 2D plane.

- Cronbachs Alpha (T1) was separate from the others because of its words (reliability, scale, item).
- Reflexivity (T0) and Triangulation (T3) were close together because they have a lot of words from qualitative research (validity, data, qualitative).

You can move a relevance parameter (λ) to see the important words that are common in the topic but not in the rest of the collection (including "denzin" and "insider").

4.3 Deployment

To bring the classification models (from Q1 & Q2) to life I made a Streamlit web app (app.py). You can paste any abstract in the text box choose a model and it will give the predicted class (AI or Classical) confidence score and the important words that make up the class.

All of this is written in the deployment/README.md so it can be run locally or started with Docker quickly.

---

5. Final Thoughts

This project was a job, with real world text data! The important thing I learned was how important text denoising is when garbage in, garbage out! The best classification result came from the model which shows that even with very little data, really clean and meaningful topics can be found if the data is properly cleaned.